LLM Security Gatekeeper

Detect adversarial prompts before they reach production LLMs

- Research pipeline for prompt injection + jailbreak detection
- Current repo state: classic ML, DeBERTa binary gate, LLM/hybrid routing, risk model for abstain resolution, and external baseline benchmarking
- Goal: maximize adversarial catch-rate without making benign UX unusable

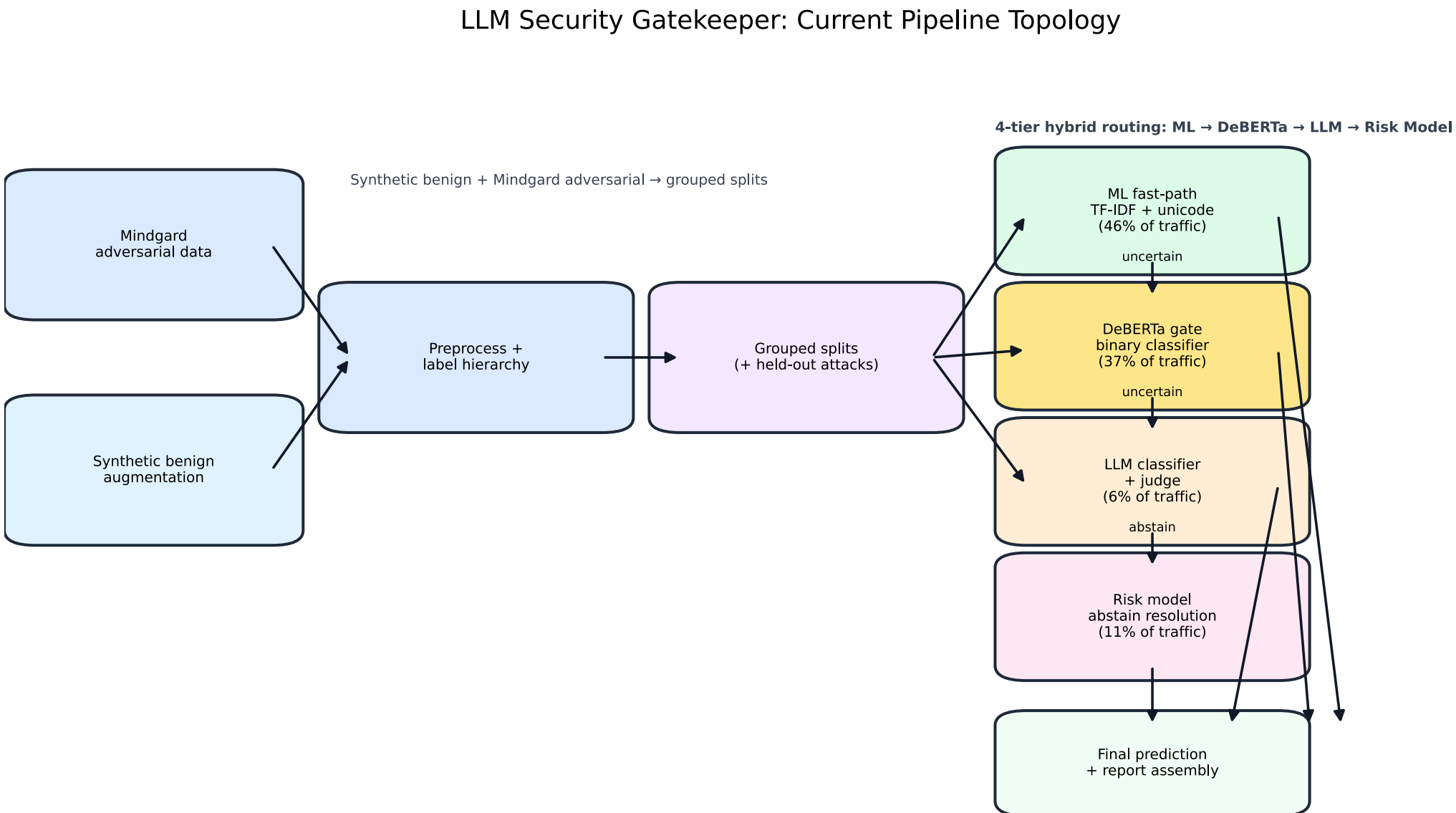
The Problem

- We need to classify prompts as `adversarial` vs `benign` before model execution
- Missed attacks are direct security failures
- False positives still matter:
  - they block legitimate user intent
  - they reduce trust in the guardrail
- The project also tracks attack category/type for analysis, not just block/allow

Why This Is Hard

- Attackers deliberately obfuscate prompts:
  - unicode tricks
  - paraphrase / typo / jailbreak variants
- Distribution shift is severe across public external datasets
- The data is highly imbalanced toward adversarial samples
- A production system needs a latency/cost-aware policy, not just one best offline score

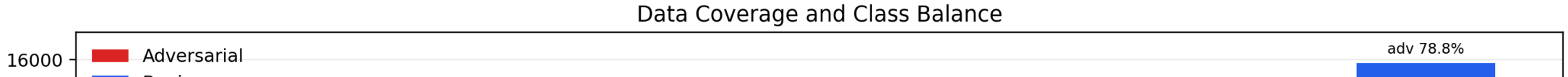
Current Pipeline



Data Construction and Coverage

Dataset/Split	Rows	Adversarial	Benign	Adv %
train	8,881	7,486	1,395	84.3%
val	1,847	1,548	299	83.8%
test	1,618	1,318	300	81.5%
test_unseen	820	820	0	100.0%

- Benigns include synthetic benign augmentation (validated via heuristic + judge + dedup)
- Held-out attack types still support unseen-attack testing
- External datasets vary sharply in size and class balance



Modeling Paths

- ML baseline:
  - TF-IDF char n-grams + handcrafted unicode features
  - Strong specialist for unicode-heavy attack patterns; fast-path in hybrid
- DeBERTa binary gate:
  - Fine-tuned `microsoft/deberta-v3-base` for binary classification
  - Primary binary gate in hybrid router (handles 37% of test samples)
  - 98.67% benign recall, 83.08% adversarial recall standalone
- LLM + hybrid:
  - LLM classifier/judge path for uncertain samples
  - Only 6% of test samples escalate to LLM (93% API call reduction)
- Risk model:
  - Logistic regression on DeBERTa probability + logprob margin + trace features

DeBERTa as Hybrid Gate

DeBERTa as Hybrid Binary Gate

Model: microsoft/deberta-v3-base

Role in hybrid router

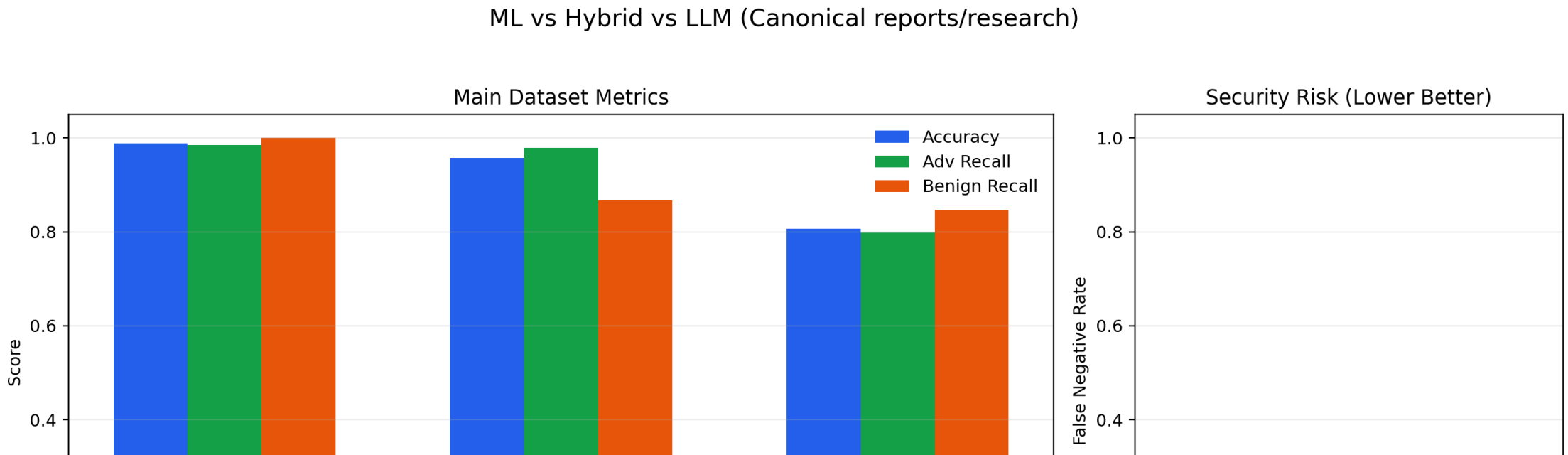
- Primary binary gate at confidence threshold 0.93
- High-confidence benign/adversarial finalized without LLM
- Handles 37% of test traffic; reduced LLM calls by 93%

Standalone performance (test)

- Accuracy: 85.97%, F1: 0.9061, ROC-AUC: 0.9890
- Benign recall: 98.67% (critical for false-positive reduction)
- Best checkpoint: epoch 3, F1 0.974

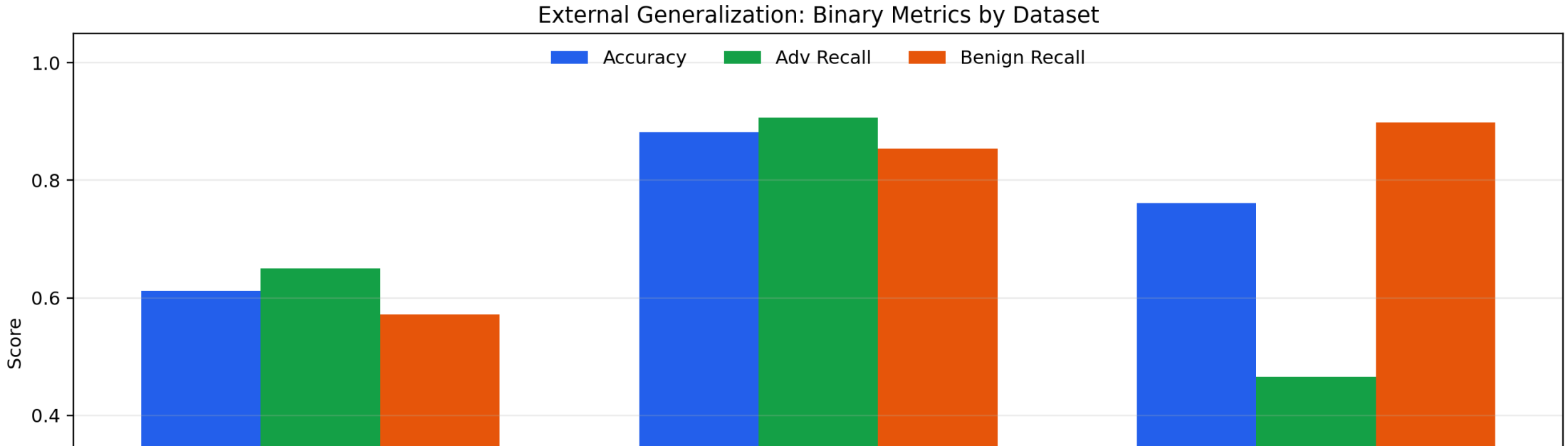
Main Results From Canonical Reports

Mode	n	Accuracy	Adv Recall	Benign Recall	FNR
ML (unicode scope)	1,070	0.9888	0.9844	1.0000	0.0156
Hybrid	1,618	0.9580	0.9788	0.8667	0.0212
LLM	1,618	0.8072	0.7982	0.8467	0.2018



External Generalization Is Still The Main Risk

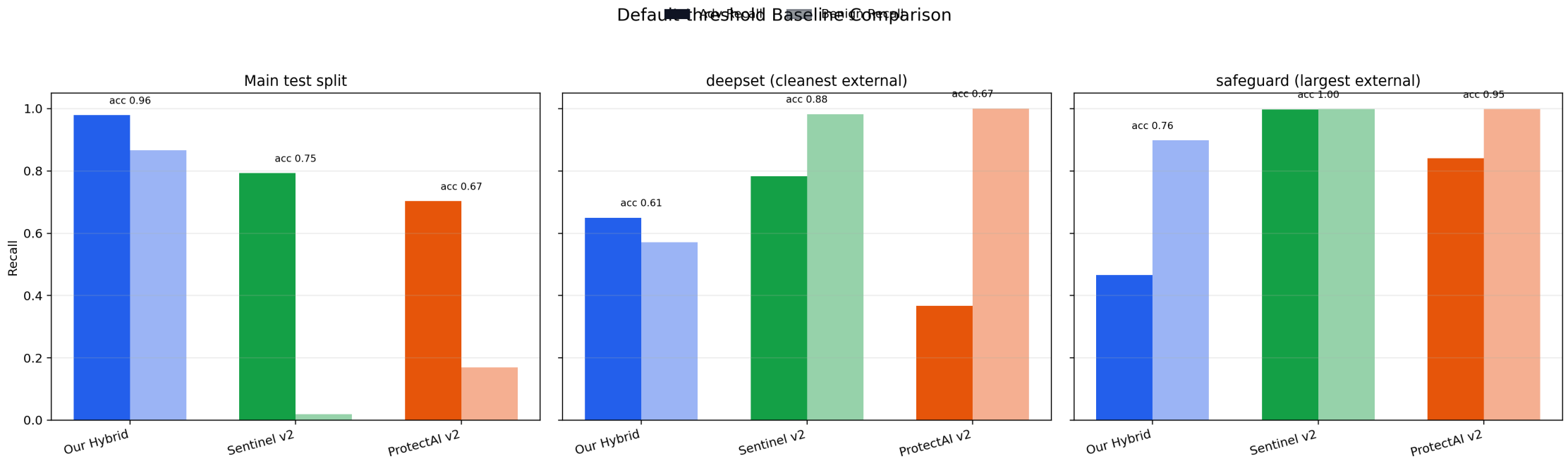
Dataset	n	Accuracy	Adv F1	Benign F1	FPR	FNR
deepset	116	0.6121	0.6341	0.5872	0.4286	0.3500
jackhhao	262	0.8817	0.8905	0.8714	0.1463	0.0935
safeguard	2,049	0.7613	0.5526	0.8373	0.1021	0.5340



External Benchmarks Need Caveats

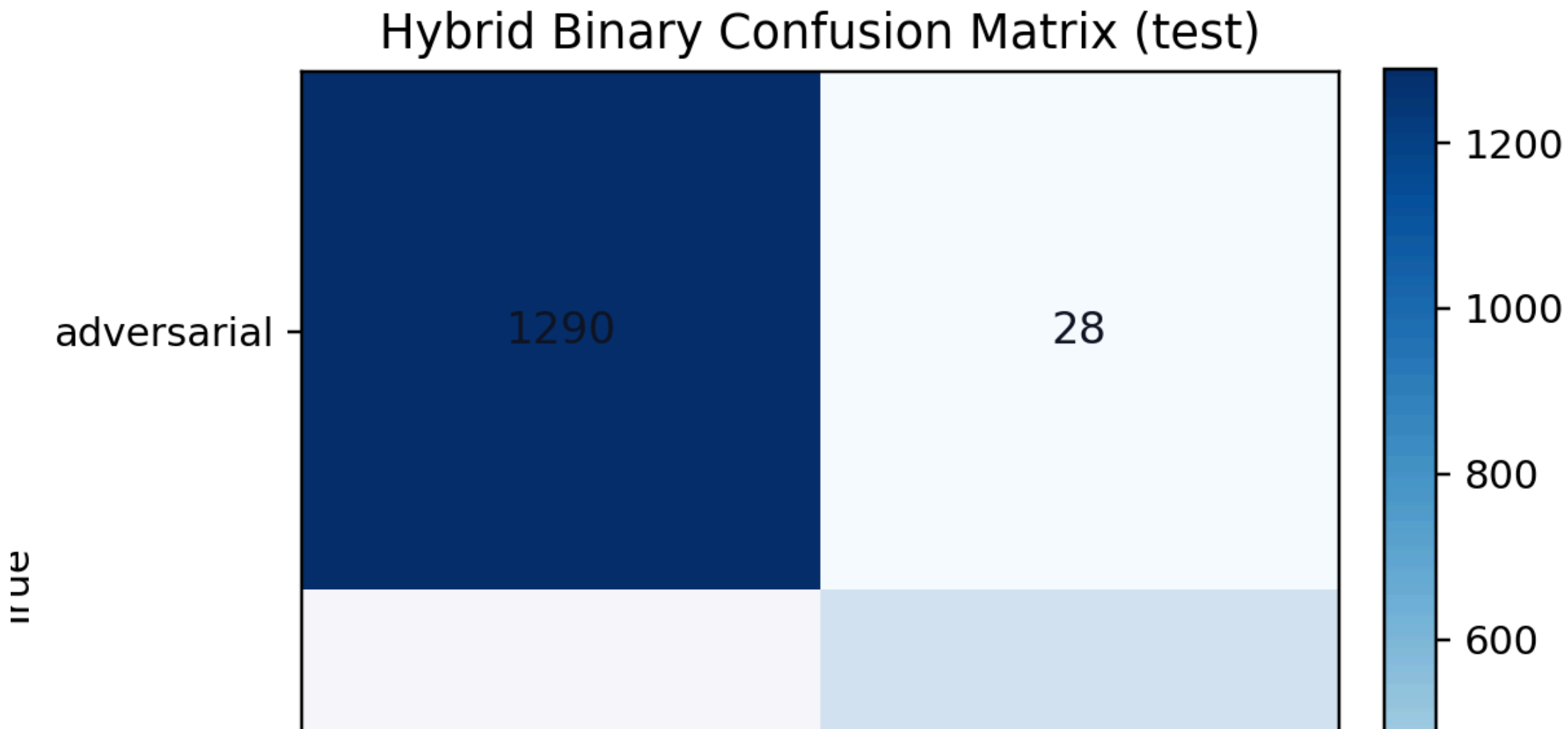
- `deepset` is the cleanest external benchmark in the repo today
- `jackhhao` has small exact overlap with our local corpus and is explicitly a ProtectAI v2 training source
- `safeguard` is not a clean unseen benchmark:
  - Mindgard is documented as Safe-Guard-derived
  - the repo overlap audit found large exact-text overlap
- So external numbers need provenance-aware interpretation, not just leaderboard reading

Baseline Comparison



- The repo benchmarks against Sentinel v2 and ProtectAI v2
- On external datasets, public baselines are much stronger (Sentinel v2: 99.8% on safeguard, 87.9% on deepset)
- On our test split, the baselines have inverted behavior: Sentinel v2 has 98% FPR

Error Pattern



Production View

- The serving shape is now concrete: 4-tier routing with measurable cost savings
  - ML: instant, handles 46% of traffic
  - DeBERTa: ~10ms inference, handles 37% of traffic
  - LLM: API call required, only 6% of traffic (93% cost reduction vs. full LLM)
  - Risk model: lightweight post-hoc, handles 11% abstain cases
- The repo already supports:
  - routing diagnostics and cost tracking
  - calibration analysis
  - reproducible report generation
  - external baseline comparison
- Still missing:
  - production API/policy engine

What Changed Recently

- DeBERTa became the primary binary gate in the hybrid router (not just a standalone model)
- Benign risk model added for post-hoc abstain resolution (ROC-AUC 0.9425)
- Hybrid accuracy jumped from ~84% to 95.8%; FNR from ~14% to 2.1%
- LLM API calls reduced by 93% through DeBERTa fast-path
- Synthetic benign data expanded (300 benigns in test, up from 137)
- Autoresearch framework added for automated routing optimization

Next Steps

1. Fix external generalization: safeguard FNR (53%) and deepset FPR (43%) are the top priorities
2. Tune risk model threshold and investigate abstain failure modes
3. Evaluate whether DeBERTa or a public baseline should anchor the low-latency path
4. Add production-facing evaluation: latency under load, cost modeling, and policy behavior
5. Explore autoresearch-driven threshold optimization across all external datasets

Appendix: Canonical Sources

- Current narrative should anchor on:
  - `reports/research/summary_report.md`
  - `reports/research/eval_report_{ml,llm,hybrid}.md`
  - `reports/deberta_classifier/summary.md`
  - `docs/2026-03-13_baseline_dataset_overlap.md`
  - `configs/default.yaml`
  - `dvc.yaml`
- DeBERTa state is backed by:
  - `src/models/deberta_classifier.py`
  - `src/cli/deberta_classifier.py`
  - `artifacts/deberta_classifier/best_checkpoint.json`
- Risk model state is backed by: